

Mise en place d'un projet Laravel avec Docker (cas pratique)

1. Pré-requis

- Docker & Docker Compose installés
- Git installé
- Accès à un terminal / IDE

Ces outils sont nécessaires pour créer, dockeriser et partager un projet Laravel.

2. Étape 1 : Créer le projet Laravel (Développeur A)

```
composer create-project laravel/laravel laravel-docker-demo
cd laravel-docker-demo
git init
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin <ton-repo-github>
git push -u origin main
```

Explication ligne par ligne :

- **composer create-project ...** : génère un projet Laravel vierge
- **cd laravel-docker-demo** : navigue dans le dossier du projet
- **git init** : initialise un dépôt Git local
- **git add .** : ajoute tous les fichiers au suivi
- **git commit -m ...** : enregistre un commit initial

- **git branch -M main** : renomme la branche en main
- **git remote add origin ...** : connecte le dépôt local à un repo distant
- **git push -u origin main** : pousse le code dans le repo distant

3. Étape 2 : Dockeriser Laravel

Dockerfile et docker-compose.yml avec commentaires ligne par ligne.

Exemple :

```
FROM php:8.2-fpm # Utilise l'image PHP 8.2 FPM
WORKDIR /var/www/html # Définit le répertoire de travail
COPY . . # Copie les fichiers locaux dans le conteneur
RUN docker-php-ext-install pdo pdo_mysql # Installe les extensions PHP
nécessaires
EXPOSE 9000 # Expose le port 9000
CMD ["php-fpm"] # Commande par défaut
```

4. Étape 3 : Lancer l'environnement Docker

```
docker-compose up -d --build
```

5. Étape 4 : Configurer Laravel

```
cp .env.example .env
docker-compose exec app php artisan key:generate
docker-compose exec app php artisan serve --host=0.0.0.0 --port=9000
```

6. Étape 5 : Développeur B récupère le projet

```
git clone <ton-repo-github>
cd laravel-docker-demo
docker-compose up -d --build
docker-compose exec app composer install
docker-compose exec app php artisan key:generate
```

7. Bonnes pratiques

- **Git** : ne pas inclure vendor/ ni .env
- **Docker Compose** : assurer la consistance des environnements
- **.env.example** : tenir à jour

8. Explications & conseils

- Docker garantit un environnement identique pour tous les développeurs
- Les volumes permettent la synchronisation instantanée
- Le Dockerfile est extensible
- Adapter Dockerfile et Compose pour la production

Conclusion : Workflow simple et collaboratif pour Docker + Laravel